

Thế lệ

Thể lệ



GDGoC-HCMUS AI Challenge 2026

Câu lạc bộ **GDGoC - HCMUS** hân hạnh giới thiệu **AI Challenge 2026** - một sân chơi học thuật sáng tạo dành cho các bạn sinh viên đam mê **Trí tuệ Nhân tạo**. Đây là cơ hội để người tham gia thử thách bản thân, trau dồi kỹ năng và phát triển tư duy nghiên cứu trong lĩnh vực AI.

Chủ đề của năm nay là **Học Tăng Cường (Reinforcement Learning)**.

1. Thông tin cuộc thi

1.1. Mục tiêu

Xây dựng các agent tự động có khả năng đưa ra chiến thuật đặt bom, né tránh nguy hiểm trong một môi trường đối kháng nhiều đội, đưa ra quyết định chiến thuật theo thời gian nhằm sinh tồn và loại bỏ đối thủ.

1.2. Hình thức thi

Cuộc thi mô phỏng trò chơi tuổi thơ **Bom IT**

- Mỗi đội gồm tối đa **2 thành viên**
- Mỗi trận đấu gồm **4 đội**, mỗi đội điều khiển 1 agent (tổng cộng 4 agent)
- Agent có thể thực hiện các hành động:
 - + Di chuyển
 - + Đặt bom
 - + Phá thùng
 - + Thu thập vật phẩm hỗ trợ

Điều kiện kết thúc trận đấu:

- Chỉ còn 1 đội duy nhất còn agent trên bản đồ, hoặc
- Hết thời gian quy định của trận đấu

1.3. Cách thức tham gia

- Đăng ký theo đội (tối đa 2 người)
-

2. Quy trình thi

Vòng 1 - Huấn luyện & Thi đấu (21/5 - 21/6)

BTC cung cấp cho thí sinh:

- Tài liệu hướng dẫn (bao gồm baseline agent, hướng dẫn tương tác với môi trường, tài liệu học Reinforcement Learning)
- Bộ công cụ phát triển (**participant kit**): engine, visualizer, v.v.
- Hướng dẫn sử dụng các nền tảng như Kaggle/Colab để huấn luyện mô hình

Trong thời gian này:

- BTC đóng vai trò mentor và hỗ trợ các đội
- Các đội có thể nộp bài nhiều lần
- Agent sẽ được tự động thi đấu với các đội khác/baseline agents để cập nhật **bảng xếp hạng**

Vòng 2 - Pitching online (24/6)

- Top 5 đội có thứ hạng cao nhất vòng Grand Finals sẽ trình bày mô hình và phương pháp của mình
-

3. Yêu cầu kỹ thuật

3.1. Thuật toán

- Không giới hạn phương pháp (Reinforcement Learning, rule-based, hybrid, v.v.)
- **Không được phép sử dụng API gọi đến các mô hình bên ngoài** trong quá trình thi đấu

3.2. Công nghệ

-
- Khuyến khích sử dụng: PyTorch, TensorFlow

3.3 Thách thức

- Môi trường chơi phức tạp, không gian tìm kiếm rộng lớn, và nhiều agent cùng tồn tại, đòi hỏi agent phải tương tác và phản ứng linh hoạt trước các hành vi khác nhau
 - Ra quyết định nhanh nhưng vẫn đảm bảo chiến thuật hiệu quả, do số lượng trạng thái lớn và sự đa dạng trong chiến thuật của các team đối thủ.
-

4. Thông tin tổ chức

-  Link đăng ký: <https://forms.gle/E3N6JWXLBsZXhRP26>
-  Thời gian mở đăng ký: 21/5.
-  Hình thức thi: Team đối đa 2 người.
-  Tổng giải thưởng: 1.500.000 đồng.
-  Hỗ trợ và hướng dẫn: #Tổng quan

Tổng quan

Tổng quan

Đạt, Tuấn

Mục lục

1. Tổng quan về trò chơi
2. Cơ chế game chi tiết
3. Đăng ký & Nộp bài
4. Cấu trúc Agent & Yêu cầu nộp bài
5. Hệ thống chấm điểm & Bảng xếp hạng
6. Các Baseline Agent
7. Tài liệu RL
8. Giải thưởng & Timeline
9. Vòng chung kết (Grand Finals)
10. Cấu hình máy chấm
11. Participant Kit
12. Tính minh bạch

1. Tổng quan về trò chơi

Bomberland là game chiến thuật theo lượt lấy cảm hứng từ Bomberman. 4 agent thi đấu trên bản đồ lưới **13×13** (vùng chơi hiệu dụng 11×11). Nhiệm vụ: di chuyển, đặt bom, phá hộp, thu thập vật phẩm và tiêu diệt đối thủ.

Bản đồ

- Kích thước: **13×13** ô (viên ngoài là tường cố định, vùng chơi 11×11).
- Sinh ngẫu nhiên mỗi trận theo **seed** - đảm bảo **kết nối toàn bộ** ô (không có vùng bị cô lập).
- Gồm 4 loại ô:

Ký hiệu	Tên	Mô tả
0	Grass (Cỏ)	Đi được, đặt bom được

Ký hiệu	Tên	Mô tả
1	Wall (Tường)	Không phá được, chặn vụ nổ
2	Box (Hộp)	Có thể phá bằng bom; khi phá có xác suất rơi vật phẩm
3	Item: Radius	Thu thập để tăng bán kính bom +1
4	Item: Capacity	Thu thập để tăng số bom mang theo +1

Vị trí xuất phát

4 agent xuất phát tại **4 góc** của bản đồ (vị trí cố định theo ID 0–3):

- Agent 0: (1, 1) - góc trên trái
- Agent 1: (11, 11) - góc dưới phải
- Agent 2: (1, 11) - góc trên phải
- Agent 3: (11, 1) - góc dưới trái

Vùng 2×2 xung quanh mỗi góc luôn được đảm bảo trống (không có hộp, không có tường) để agent không bị kẹt ngay từ đầu.

2. Cơ chế game chi tiết

Thứ tự xử lý mỗi step

Mỗi step game xử lý theo thứ tự sau:

Thu thập hành động → Xử lý di chuyển → Đặt bom → Giảm timer bomb → Kích nổ bom có time = 0 → Loại agent → Spawn vật phẩm → Kiểm tra kết thúc

Hành động của Agent

Mỗi step, agent trả về **1 số nguyên**:

Giá trị	Hành động
0	STOP (đứng yên)
1	LEFT (trái) → UP
2	RIGHT (phải) → DOWN
3	UP (lên) → LEFT
4	DOWN (xuống) → RIGHT
5	PLACE_BOMB (đặt bom)

Lưu ý (5/6/2026): Bug ở hàm `step()`, trong engine/[game.py](#). Hiện tại hệ thống quy định `x` làm hàng, `y` làm cột nhưng code xử lý phần di chuyển đã cập nhật sai. Vì cuộc thi đang diễn ra, BTC sẽ không thay đổi engine code, tránh phá vỡ logic bên trong các agent đã được nộp, mà sẽ cập nhật lại tài liệu để phản ánh đúng hành động sẽ được thực hiện (bảng trên).

Quy tắc di chuyển:

- Không đi xuyên tường (1) hoặc hộp (2).
- **Không đi xuyên bom** của step trước (bom đã tồn tại trên sàn). Ngoại lệ: agent đang đứng đúng ô đó khi đặt bom thì vẫn có thể rời đi.
- Nhiều agent **có thể đứng trên cùng một ô**.
- Nếu 2 agent cùng bước vào ô có vật phẩm đồng thời → vật phẩm bị hủy (không ai nhận).

Cơ chế Bom

Thông số mặc định:

- Timer: **7 steps** (đếm từ 7 xuống 0, nổ khi ≤ 0)
- Bán kính mặc định: **1** (tăng theo item thu thập được)
- Số bom ban đầu: **1** (tăng theo item)
- Bán kính tối đa: **5**, Số bom đặt tối đa: **5**

Quy tắc đặt bom:

- Chỉ đặt được nếu `bombs_left > 0`.
- Không đặt lên ô đã có bom từ step trước.

- Nếu nhiều agent đặt bom tại cùng một ô trong **cùng một step** → engine xử lý agent theo thứ tự ID tăng dần ($0 \rightarrow 3$). Bom của agent sau **chỉ ghi đè** bom trước nếu có bán kính **lớn hơn**. Kết quả cụ thể:
 - Bán kính khác nhau → đặt bom có **bán kính lớn hơn** (bất kể agent ID nào).
 - Bán kính bằng nhau → đặt bom của **agent có ID nhỏ hơn**
 - Chỉ **agent sở hữu bom được đặt** mới bị trừ **bombs_left**.
- Chỉ **người đặt bom** mới tiêu tốn **bombs_left**. Bom vẫn tồn tại và chờ nổ kể cả khi agent đó đã bị tiêu diệt.

Cơ chế nổ:

- Bom nổ theo **4 hướng** (lên, xuống, trái, phải).
- Vụ nổ **dừng tại tường** (không xuyên qua).
- Vụ nổ **dừng tại hộp** và **phá hộp** đó (không xuyên tiếp).
- Vụ nổ **xuyên qua agent** (không bị chặn bởi người chơi).
- Vụ nổ chỉ kéo dài **1 step**.
- **Chain reaction**: Nếu vụ nổ chạm vào một bom khác, bom đó nổ ngay lập tức trong cùng step đó.

Khi hộp bị phá:

- 30% xác suất rơi **Item Radius** (+1 bán kính bom)
- 30% xác suất rơi **Item Capacity** (+1 bom mang theo)
- 40% không rơi gì

Vật phẩm tự spawn: Mỗi step, mỗi ô cỏ trống (không có agent đứng) có xác suất nhỏ tự spawn vật phẩm:

- Xác suất tăng dần theo số step đã qua: $P = 0.0003 \times (\text{step} / 165)$
- 50% là Radius, 50% là Capacity

Khi agent bị tiêu diệt:

- Bất kỳ agent nào đứng trên ô bị ảnh hưởng bởi vụ nổ **bị loại ngay lập tức**.
- Agent bị loại sẽ không thể hành động thêm. Tuy nhiên, các quả bom mà agent đó đã đặt trước khi bị tiêu diệt vẫn tồn tại trên bản đồ và sẽ tiếp tục đếm ngược (timer) để nổ như bình thường.

Tự hủy (Self-destruct): Agent có thể tự tiêu diệt bản thân

Kết thúc trận đấu

Trận kết thúc khi:

- Còn **≤ 1 agent sống sót**, hoặc
- Đạt **500 steps**

3. Đăng ký & Nộp bài

Bước 1: Đăng ký đội

Điền [Form Đăng ký](#) với các thông tin:

- Tên đội (phải khác các đội khác)
- Tên & email người liên hệ chính
- Tên & email người liên hệ phụ (nếu có)
- Đồng ý với quy định cuộc thi

Sau khi đăng ký thành công, hệ thống sẽ gửi **email xác nhận** về địa chỉ của người liên hệ chính, bao gồm:

Thông tin	Mô tả
Team Name	Tên đội đã đăng ký
Canonical Team ID	ID định danh duy nhất (dùng khi nộp bài)
Submission Token	Mật khẩu xác thực khi nộp bài (không chia sẻ)
Link Form Nộp bài	Form dùng để nộp agent
Link Discord/Liên hệ	Kênh hỗ trợ

Đăng ký lại cùng (tên đội, email chính) sẽ tạo ra **Submission Token** mới và vô hiệu hóa token cũ.

Lưu ý: Sau khi đăng ký hợp lệ nhưng không có email xác nhận, có khả năng đã có một đội khác trùng tên đã đăng ký từ trước, lúc này hãy đăng ký đội lại với tên khác.

Bước 2: Nộp bài

Điền **Form Nộp bài** với:

- Canonical Team ID (từ email)
- Submission Token (từ email)
- Mô tả thay đổi (changelog) (optional)
- File .zip

Hạn mức nộp bài: 3 lần/ngày (reset lúc 7:00 sáng giờ Việt Nam).

4. Cấu trúc Agent & Yêu cầu nộp bài

Cấu trúc file ZIP

File nộp bài phải là một file .zip chứa **đúng một file agent.py** ở bất kỳ cấp thư mục nào:

submission.zip

- └─ agent.py ← Bắt buộc, duy nhất
- └─ model.pth ← (tùy chọn, nếu dùng RL)
- └─ ... ← (tùy chọn, các file weights, config nhỏ lẻ khác)

Giới hạn file

Tiêu chí	Giới hạn
Kích thước file ZIP	≤ 100 MB
Tổng kích thước sau giải nén	≤ 300 MB
Kích thước một file đơn lẻ	≤ 150 MB
Số file tối đa	≤ 20 file
Phần mở rộng được phép	.py, .txt, .pt, .pth, .pkl, .onnx, .bin, .json, .yaml, .yml, .md, .h5, .pb, .keras, .tflite

Class Agent bắt buộc

File `agent.py` **phải** định nghĩa class `Agent` với đúng interface sau:

```
class Agent:
    def __init__(self, agent_id: int):
        # agent_id: 0, 1, 2, hoặc 3, được tự động gán bởi engine
        self.agent_id = agent_id

    def act(self, obs: dict) -> int:
        # obs: dict gồm 'map', 'players', 'bombs'
        # Trả về: int trong [0, 5]
        ...
```

Observation (obs) nhận được mỗi step

```
obs = {
    "map": np.ndarray, # shape (13, 13), dtype int
            # 0=Grass, 1=Wall, 2=Box, 3=Item_Radius, 4=Item_Capacity
    "players": np.ndarray, # shape (4, 5), dtype int8
            # Mỗi hàng: [row, col, alive, bombs_left, bomb_radius_bonus]
    "bombs": np.ndarray, # shape (N, 4), dtype int8, N = số bom hiện có
            # Mỗi hàng: [row, col, timer, owner_id]
}
```

Lưu ý:

- `alive`: 1 = còn sống, 0 = đã chết
- `bomb_radius_bonus`: số bonus thêm vào (bán kính thực = 1 + bonus)
- `bombs_left`: số bom có thể đặt thêm
- Agent nhận **full state** như trên

Yêu cầu về thời gian

- **Startup timeout**: 20 giây
- **Inference timeout**: `act()` phải trả về trong vòng **100ms**. Nếu không, hệ thống mặc định agent thực hiện hành động 0 (STOP).

Quy định quan trọng

- ❌ **Không được dùng LLM** (GPT, Gemini, Claude,...) bên trong class `Agent`.
- ❌ **Không được copy code** từ đội khác.

-
-  **Không được import thêm thư viện không có sẵn** trong môi trường chấm (xem requirements.txt ở Participant Kit).
 -  Được phép load model từ file checkpoints đi kèm trong ZIP.
 -  Mỗi bài nộp được coi là một agent riêng biệt và dùng một lần nộp trong ngày.
-

5. Hệ thống chấm điểm & Bảng xếp hạng

Cách chấm điểm: TrueSkill

Hệ thống sử dụng thuật toán [TrueSkill](#) (Microsoft Research) để ước tính kỹ năng thực sự của từng agent. Mỗi submission được đặc trưng bởi:

- **μ (mu):** Ước tính kỹ năng trung bình
- **σ (sigma):** Độ không chắc chắn (sigma giảm khi chơi nhiều trận hơn)
- **Score = $\mu - 3\sigma$:** Điểm thật, dùng để xếp hạng - đảm bảo các agent đã chơi đủ nhiều trận mới được xếp hạng cao

Khởi đầu mặc định: $\mu = 100$, $\sigma = 33.3333$.

Kết quả mỗi trận

Mỗi trận 4 agent. Thứ hạng trong trận được xác định theo **thứ tự bị tiêu diệt**:

- Agent bị tiêu diệt cuối cùng (hoặc còn sống đến hết game) = hạng tốt nhất
- Agent bị tiêu diệt sớm nhất = hạng tệ nhất
- Nhiều agent bị tiêu diệt cùng step $\rightarrow$ cùng hạng (draw)

Tie-break ở Step 500:

Nếu có nhiều agent còn sống tới step 500, các agent này sẽ không còn mặc định được tính là có cùng hạng tốt nhất nữa. Thay vào đó, thứ hạng giữa các agent còn sống sẽ được xác định dựa trên mức độ tham gia giao tranh và tương tác trong trận đấu, theo thứ tự ưu tiên sau:

1. Số lượng tiêu diệt (Kills)
2. Số lượng hộp phá hủy (Boxes Destroyed)
3. Số lượng vật phẩm nhặt được (Items Collected)
4. Số lượng bom đã đặt (Bombs Placed)

Các agent có các giá trị hoàn toàn giống nhau ở cả 4 tiêu chí trên sẽ được tính là hòa.

Win/Draw/Loss:

- **Win:** Khi hạng của agent là tốt nhất và là duy nhất.
- **Draw:** Cùng thứ hạng tốt nhất với các agent khác.
- **Loss:** Không đạt được hạng tốt nhất.

Bảng xếp hạng (Leaderboard)

Thứ tự ưu tiên xếp hạng (tie-break):

1. **Score** = $\mu - 3\sigma$ **cao hơn** → xếp trước
2. Nếu bằng: μ **cao hơn** → xếp trước
3. Nếu vẫn bằng: σ **thấp hơn** → xếp trước
4. Nếu vẫn bằng: **nộp bài gần đây hơn** → xếp trước

Hệ thống đánh giá tự động**Sau khi nộp:**

1. Server tải file ZIP từ Google Drive
2. Kiểm tra cấu trúc và cú pháp, chạy thử một trận
3. Nếu hợp lệ → chạy ngay **một batch 12 trận** với các agent trong pool. Đối thủ được chọn từ Active Pool (với tỉ lệ: 40% điểm gần giống, 30% top cao nhất, 30% ngẫu nhiên).
4. Cập nhật bảng xếp hạng Google Sheets

Background job (liên tục):

- Worker chạy liên tục theo chu kỳ: mỗi chu kỳ chạy **5 trận** rồi nghỉ 10 giây rồi lặp lại.
- Mỗi trận được chọn từ **Active Pool** (xem bên dưới), đảm bảo ít nhất 1 agent không phải là baseline tham gia.

Active Pool

Gồm:

Tiêu chí	Mô tả
Tất cả Baseline	Luôn trong pool
Best per Team	Submission tốt nhất (theo Score) của mỗi đội
Recent per Team	2 submission gần nhất của mỗi đội

Tiêu chí	Mô tả
Top Global	Top 10 submission toàn cuộc thi (≥ 10 trận)

6. Các Baseline Agent

Hệ thống có **6 rule-based baseline agent** với rating thay đổi. Bảng sau đây là rating ban đầu khi cho các baselines tự đấu với nhau 1000+ trận:

Tên	Chiến lược	Score ($\mu - 3\sigma$)
tactical_rule_agent	Né nguy hiểm, tìm vật phẩm, nhắm kẻ địch, đặt bom có tính toán	~114.7
genius_rule_agent	Cân bằng tấn công và phòng thủ, BFS tìm đường	~112.5
smarter_rule_agent	Ưu tiên phá hộp, né bom, đuổi kẻ địch	~111.3
box_farmer_agent	Tập trung phá hộp để thu item	~107.9
simple_rule_agent	Logic đơn giản: né bom, đặt bom	~107.8
random_agent	Hành động ngẫu nhiên	~99.0

Ngoài ra có **1 RL-based agent** (`dqn_agent`) làm tham khảo cách triển khai RL

7. Tài liệu RL

- RL Book, Sutton Barto: <http://incompleteideas.net/book/the-book-2nd.html>
- RL Course Toronto University: <https://bercyhi-courses.github.io/rl-utoronto/>
- Neuriton: <https://www.facebook.com/share/1B4BpDUbyx/>
- RL Exercises, AI VIETNAM: <https://www.facebook.com/share/p/192LyNsvbp/>

- Single-File RL Algorithms Code: <https://github.com/vwxyzjn/cleanrl>

8. Giải thưởng & Timeline

Giải thưởng (dự kiến)

Hạng	Giải thưởng
 1 Giải Nhất	500.000 VNĐ
 1 Giải Nhì	400.000 VNĐ
 1 Giải Ba	300.000 VNĐ
 1 Giải Tư	200.000 VNĐ
 1 Giải Khuyến Khích	100.000 VNĐ

Timeline

Mốc thời gian	Sự kiện
21/5	Mở đăng ký & nộp bài
24/5	Workshop hỏi đáp về cuộc thi & RL
21/6	Đóng nộp bài, Freeze bảng xếp hạng
22/6 00:00 → 12:00	Chạy ngầm các trận để ổn định rating
22/6 12:00	Chọn Top 8 → chạy Grand Finals → Công bố kết quả
24/6	Pitching online & Trao giải

Quy mô dự kiến: 25-30 đội (tối đa 2 người mỗi đội), mở rộng cho cả người ngoài câu lạc bộ tham gia.

Cập nhật 1/6/2026: 250 đội đã đăng ký.

9. Vòng chung kết (Grand Finals)

Chọn Top K

Sau khi freeze bảng xếp hạng, hệ thống sẽ chạy vô số các trận trong **12 tiếng đầu ngày 22/6** để ổn định rating và chốt bảng rating cuối cùng, sau đó chọn **Top 8 đội** dựa trên submission tốt nhất của mỗi đội và số trận đã chơi ≥ 50 trận, theo thứ tự ưu tiên:

1. Score ($\mu - 3\sigma$) cao nhất
2. Nếu bằng: μ cao hơn
3. Nếu vẫn bằng: σ thấp hơn

Ngoài ra sẽ có thêm **1 baseline agent tốt nhất** tham gia làm thước đo chuẩn.

Format Grand Finals: Round Robin

- Tất cả tổ hợp 4-player từ pool finalist được liệt kê ($C(9,4)$ tổ hợp).
- Mỗi tổ hợp chạy **50 trận** (shuffle vị trí góc ngẫu nhiên mỗi trận).
- Điểm tính theo hạng trong từng trận:
 - Hạng 1 (tốt nhất): **3 điểm**
 - Hạng 2: **2 điểm**
 - Hạng 3: **1 điểm**
 - Hạng 4 (tệ nhất): **0 điểm**

Tie-break Grand Finals

Nếu tổng điểm bằng nhau:

1. Score ($\mu - 3\sigma$) từ bảng xếp hạng sau 12 tiếng chấm ngấm vào 22/6 $\rightarrow$ cao hơn thắng
2. Nếu vẫn bằng: μ cao hơn thắng
3. Nếu vẫn bằng: σ thấp hơn thắng

Online pitching

Đội top 5 sau Grand Finals thuyết trình về thuật toán của đội.

10. Cấu hình máy chấm

Thông số	Chi tiết
Máy chủ	Google Cloud VM (e2-standard-8)
CPU	8 vCPU (4 core vật lý)
RAM	32 GB
Hệ điều hành	Ubuntu 22.04 LTS
Python	3.11 (Conda environment)
Inference timeout	100ms/step
Startup timeout	20 giây
Max steps/trận	500 steps

11. Participant Kit

Participant Kit là repo **công khai** chứa mọi thứ cần thiết để phát triển agent.

Link repo: [here](#).

12. Tính minh bạch

Seed cố định

Mỗi trận được gán **seed ngẫu nhiên duy nhất** khi chạy. Seed này được lưu trong log JSON → đảm bảo minh họa lại trận đấu đó bằng cách chạy engine với cùng seed và cùng agent.

Logs công khai

Mỗi trận được lưu thành:

-
- **File JSON:** Chứa toàn bộ lịch sử mỗi step (map, players, bombs, actions), seed, xếp hạng.
 - **File GIF:** Animation trận đấu.

Các file này được upload lên [Google Drive](#) sau mỗi trận và link được ghi vào Google Sheets để có thể truy cập trực tiếp.

Bảng xếp hạng

Bảng xếp hạng được cập nhật lên [Google Sheets](#) ngay sau mỗi lần nộp bài và thường xuyên trong ngày. Ngoài ra, Google Sheets còn dùng để hiển thị logs khi nộp bài.

Repo công khai

Toàn bộ source code engine game và các baseline agent được public trên GitHub, đảm bảo mọi người có thể kiểm tra logic game.

Tài liệu này sẽ được cập nhật khi có thay đổi. Mọi thắc mắc liên hệ qua Discord hoặc email ban tổ chức.